

Go from Data to Strategy ▶

ONLINE MASTER OF SCIENCE IN
BUSINESS ANALYTICSCarnegie Mellon University
Tepper School of Business

Go from Data to Strategy: Tepper School of Business

How to Evaluate the Performance of PyTorch Models

by **Adrian Tam** on [April 8, 2023](#) in **Deep Learning with PyTorch** 💬 6

Share

Share

Post

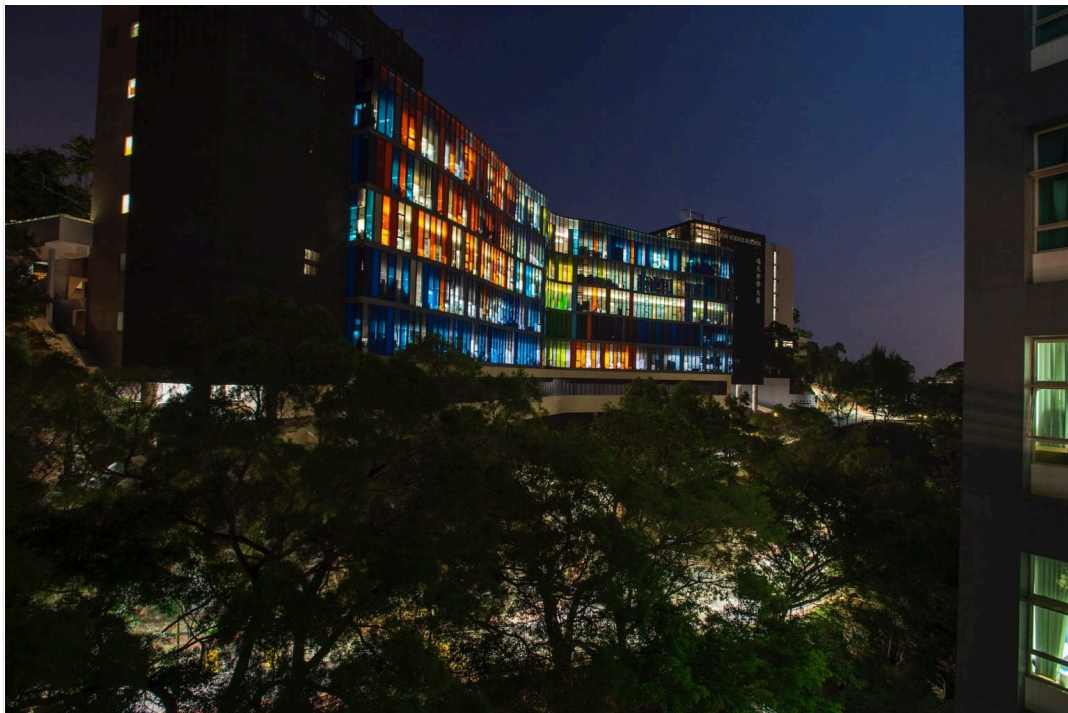
Designing a deep learning model is sometimes an art. There are a lot of decision points, and it is not easy to tell what is the best. One way to come up with a design is by trial and error and evaluating the result on real data. Therefore, it is important to have a scientific method to evaluate the performance of your neural network and deep learning models. In fact, it is also the same method to compare any kind of machine learning models on a particular usage.

In this post, you will discover the received workflow to robustly evaluate model performance. In the examples, we will use PyTorch to build our models, but the method can also be applied to other models. After completing this post, you will know:

- How to evaluate a PyTorch model using a verification dataset
- How to evaluate a PyTorch model with k-fold cross-validation

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



How to evaluate the performance of PyTorch models
Photo by Kin Shing Lai. Some rights reserved.

Overview

This chapter is in four parts; they are:

- Empirical Evaluation of Models
- Data Splitting
- Training a PyTorch Model with Validation

- k-Fold Cross Validation

Empirical Evaluation of Models

In designing and configuring a deep learning model from scratch, there are a lot of decisions to make. This includes design decisions such as how many layers to use in a deep learning model, how big is each layer, and what kind of layers or activation functions to use. It can also be the choice of the loss function, optimization algorithm, number of epochs to train, and the interpretation of the model output. Luckily, sometimes, you can copy the structure of other people's networks. Sometimes, you can just make up your choice using some heuristics. To tell if you made a good choice or not, the best way is to compare multiple alternatives by empirically evaluating them with actual data.

Deep learning is often used on problems that have very large datasets. That is tens of thousands or hundreds of thousands of data samples. This provides ample data for testing. But you need to have a robust test strategy to estimate the performance of your model on unseen data. Based on that, you can have a metric to compare among different model configurations.

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

Data Splitting

If you have a dataset of tens of thousands of samples or even more, you don't always need to give everything to your model for training. This will unnecessarily increase the complexity and lengthen the training time. More is not always better. You may not get the best result.

When you have a large amount of data, you should take a portion of it as the **training set** that is fed into the model for training. Another portion is kept as a **test set** to hold back from the training but verified with a trained or partially trained model as an evaluation. This step is usually called "train-test split."

Let's consider the [Pima Indians Diabetes dataset](#). You can load the data using NumPy:

```
1 import numpy as np
2 data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
```

There are 768 data samples. It is not a lot but is enough to demonstrate the split. Let's consider the first 66% as the training set and the remaining as the test set. The easiest way to do so is by slicing an array:

```
1 # find the boundary at 66% of total samples
2 count = len(data)
3 n_train = int(count * 0.66)
4 # split the data at the boundary
5 train_data = data[:n_train]
6 test_data = data[n_train:]
```

The choice of 66% is arbitrary, but you do not want the training set too small. Sometimes you may use 70%-30% split. But if the dataset is huge, you may even use a 30%-70% split if 30% of training data is large enough.

If you split the data in this way, you're suggesting the datasets are shuffled so that the training set and the test set are equally diverse. If you find the original dataset is sorted and take the test set only at the end, you may find you have all the test data belonging to the same class or carrying the same value in one of the input features. That's not ideal.

Of course, you can call `np.random.shuffle(data)` before the split to avoid that. But many machine learning engineers usually use scikit-learn for this. See this example:

```
1 import numpy as np
2 from sklearn.model_selection import train_test_split
3
4 data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
5 train_data, test_data = train_test_split(data, test_size=0.33)
```

But more commonly, it is done after you separate the input feature and output labels. Note that this function from scikit-learn can work not only on NumPy arrays but also on PyTorch tensors:

```
1 import numpy as np
2 import torch
```

```

3 from sklearn.model_selection import train_test_split
4
5 data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
6 X = data[:, 0:8]
7 y = data[:, 8]
8 X = torch.tensor(X, dtype=torch.float32)
9 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)

```

Training a PyTorch Model with Validation

Let's revisit the code for building and training a deep learning model on this dataset:

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import tqdm
5
6 ...
7
8 model = nn.Sequential(
9     nn.Linear(8, 12),
10    nn.ReLU(),
11    nn.Linear(12, 8),
12    nn.ReLU(),
13    nn.Linear(8, 1),
14    nn.Sigmoid()
15 )
16
17 # loss function and optimizer
18 loss_fn = nn.BCELoss() # binary cross entropy
19 optimizer = optim.Adam(model.parameters(), lr=0.0001)
20
21 n_epochs = 50 # number of epochs to run
22 batch_size = 10 # size of each batch
23 batches_per_epoch = len(X_train) // batch_size
24
25 for epoch in range(n_epochs):
26     with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
27         bar.set_description(f"Epoch {epoch}")
28         for i in bar:
29             # take a batch
30             start = i * batch_size
31             X_batch = X_train[start:start+batch_size]
32             y_batch = y_train[start:start+batch_size]
33             # forward pass
34             y_pred = model(X_batch)
35             loss = loss_fn(y_pred, y_batch)
36             # backward pass
37             optimizer.zero_grad()
38             loss.backward()
39             # update weights
40             optimizer.step()
41             # print progress
42             bar.set_postfix(
43                 loss=float(loss)
44             )

```

In this code, one batch is extracted from the training set in each iteration and sent to the model in the forward pass. Then you compute the gradient in the backward pass and update the weights.

While, in this case, you used binary cross entropy as the loss metric in the training loop, you may be more concerned with the prediction accuracy. Calculating accuracy is easy. You round off the output (in the range of 0 to 1) to the nearest integer so you can get a binary value of 0 or 1. Then you count how much percentage your prediction matched the label; this gives you the accuracy.

But what is your prediction? It is `y_pred` above, which is the prediction by your current model on `X_batch`. Adding accuracy to the training loop becomes this:

```

1 for epoch in range(n_epochs):
2     with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
3         bar.set_description(f"Epoch {epoch}")
4         for i in bar:
5             # take a batch
6             start = i * batch_size
7             X_batch = X_train[start:start+batch_size]
8             y_batch = y_train[start:start+batch_size]
9             # forward pass
10            y_pred = model(X_batch)
11            loss = loss_fn(y_pred, y_batch)
12            # backward pass
13            optimizer.zero_grad()
14            loss.backward()
15            # update weights
16            optimizer.step()
17            # print progress, with accuracy
18            acc = (y_pred.round() == y_batch).float().mean()
19            bar.set_postfix(
20                loss=float(loss),
21                acc=float(acc)
22            )

```

However, the `X_batch` and `y_batch` is used by the optimizer, and the optimizer will fine-tune your model so that it can predict `y_batch` from `X_batch`. And now you're using accuracy to check if `y_pred` match with `y_batch`. It is like cheating because if your model somehow remembers the solution, it can just report to you the `y_pred` and get perfect accuracy without actually inferring `y_pred` from `X_batch`.

Indeed, a deep learning model can be so convoluted that you cannot know if your model simply remembers the answer or is inferring the answer.

Therefore, the best way is **not** to calculate accuracy from `X_batch` or anything from `X_train` but from something else: your test set. Let's add an accuracy measurement **after** each epoch using `X_test`:

```

1 for epoch in range(n_epochs):
2     with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
3         bar.set_description(f"Epoch {epoch}")
4         for i in bar:
5             # take a batch
6             start = i * batch_size
7             X_batch = X_train[start:start+batch_size]
8             y_batch = y_train[start:start+batch_size]
9             # forward pass
10            y_pred = model(X_batch)
11            loss = loss_fn(y_pred, y_batch)
12            # backward pass
13            optimizer.zero_grad()
14            loss.backward()
15            # update weights
16            optimizer.step()
17            # print progress
18            acc = (y_pred.round() == y_batch).float().mean()
19            bar.set_postfix(
20                loss=float(loss),
21                acc=float(acc)
22            )
23        # evaluate model at end of epoch
24        y_pred = model(X_test)
25        acc = (y_pred.round() == y_test).float().mean()
26        acc = float(acc)
27        print(f"End of {epoch}, accuracy {acc}")

```

In this case, the `acc` in the inner for-loop is just a metric showing the progress. Not much difference in displaying the loss metric, except it is not involved in the gradient descent algorithm. And you expect the accuracy to improve as the loss metric also improves.

In the outer for-loop, at the end of each epoch, you calculate the accuracy from `X_test`. The workflow is similar: You give the test set to the model and ask for its prediction, then count the number of matched results with your test set labels. But this accuracy is the one you should care about. It should improve as the training progresses, but if you do not see it improve (i.e., accuracy increase) or even deteriorates, you have to interrupt the training as it seems to start overfitting. Overfitting is when the model started to remember the training set rather than learning to infer the prediction from it. A sign of that is the accuracy from the training set keeps increasing while the accuracy from the test set decreases.

The following is the complete code to implement everything above, from data splitting to validation using the test set:

```

1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 import tqdm
6 from sklearn.model_selection import train_test_split
7
8 data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
9 X = data[:, 0:8]
10 y = data[:, 8]
11 X = torch.tensor(X, dtype=torch.float32)
12 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
13 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
14
15 model = nn.Sequential(
16     nn.Linear(8, 12),
17     nn.ReLU(),
18     nn.Linear(12, 8),

```

```

19     nn.ReLU(),
20     nn.Linear(8, 1),
21     nn.Sigmoid()
22 )
23
24 # loss function and optimizer
25 loss_fn = nn.BCELoss() # binary cross entropy
26 optimizer = optim.Adam(model.parameters(), lr=0.0001)
27
28 n_epochs = 50 # number of epochs to run
29 batch_size = 10 # size of each batch
30 batches_per_epoch = len(X_train) // batch_size
31
32 for epoch in range(n_epochs):
33     with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar: #, disable=True) as bar:
34         bar.set_description(f"Epoch {epoch}")
35         for i in bar:
36             # take a batch
37             start = i * batch_size
38             X_batch = X_train[start:start+batch_size]
39             y_batch = y_train[start:start+batch_size]
40             # forward pass
41             y_pred = model(X_batch)
42             loss = loss_fn(y_pred, y_batch)
43             # backward pass
44             optimizer.zero_grad()
45             loss.backward()
46             # update weights
47             optimizer.step()
48             # print progress
49             acc = (y_pred.round() == y_batch).float().mean()
50             bar.set_postfix(
51                 loss=float(loss),
52                 acc=float(acc)
53             )
54         # evaluate model at end of epoch
55         y_pred = model(X_test)
56         acc = (y_pred.round() == y_test).float().mean()
57         acc = float(acc)
58         print(f"End of {epoch}, accuracy {acc}")

```

The code above will print the following:

```

1 End of 0, accuracy 0.5787401795387268
2 End of 1, accuracy 0.6102362275123596
3 End of 2, accuracy 0.6220472455024719
4 End of 3, accuracy 0.6220472455024719
5 End of 4, accuracy 0.6299212574958801
6 End of 5, accuracy 0.6377952694892883
7 End of 6, accuracy 0.6496062874794006
8 End of 7, accuracy 0.6535432934761047
9 End of 8, accuracy 0.665354311466217
10 End of 9, accuracy 0.6614173054695129
11 End of 10, accuracy 0.665354311466217
12 End of 11, accuracy 0.665354311466217
13 End of 12, accuracy 0.665354311466217
14 End of 13, accuracy 0.665354311466217
15 End of 14, accuracy 0.665354311466217
16 End of 15, accuracy 0.6732283234596252
17 End of 16, accuracy 0.6771653294563293
18 End of 17, accuracy 0.6811023354530334
19 End of 18, accuracy 0.6850393414497375
20 End of 19, accuracy 0.6889764070510864
21 End of 20, accuracy 0.6850393414497375
22 End of 21, accuracy 0.6889764070510864
23 End of 22, accuracy 0.6889764070510864
24 End of 23, accuracy 0.6889764070510864
25 End of 24, accuracy 0.6889764070510864
26 End of 25, accuracy 0.6850393414497375
27 End of 26, accuracy 0.6811023354530334
28 End of 27, accuracy 0.6771653294563293
29 End of 28, accuracy 0.6771653294563293
30 End of 29, accuracy 0.6692913174629211
31 End of 30, accuracy 0.6732283234596252
32 End of 31, accuracy 0.6692913174629211
33 End of 32, accuracy 0.6692913174629211
34 End of 33, accuracy 0.6732283234596252
35 End of 34, accuracy 0.6771653294563293
36 End of 35, accuracy 0.6811023354530334
37 End of 36, accuracy 0.6811023354530334
38 End of 37, accuracy 0.6811023354530334
39 End of 38, accuracy 0.6811023354530334
40 End of 39, accuracy 0.6811023354530334
41 End of 40, accuracy 0.6811023354530334
42 End of 41, accuracy 0.6771653294563293
43 End of 42, accuracy 0.6771653294563293
44 End of 43, accuracy 0.6771653294563293
45 End of 44, accuracy 0.6771653294563293
46 End of 45, accuracy 0.6771653294563293
47 End of 46, accuracy 0.6771653294563293
48 End of 47, accuracy 0.6732283234596252
49 End of 48, accuracy 0.6732283234596252
50 End of 49, accuracy 0.6732283234596252

```

k-Fold Cross Validation

In the above example, you calculated the accuracy from the test set. It is used as a **score** for the model as you progressed in the training. You want to stop at the point where this score is at its maximum. In fact, by merely comparing the score from this test set, you know your model works best after epoch 21 and starts to overfit afterward. Is that right?

If you built two models of different designs, should you just compare these models' accuracy on the same test set and claim one is better than another?

Actually, you can argue that the test set is not representative enough even after you have shuffled your dataset before extracting the test set. You may also argue that, by chance, one model fits better to this particular test set but not always better. To make a stronger argument on which model is better independent of the selection of the test set, you can try **multiple test sets** and average the accuracy.

This is what a k-fold cross validation does. It is a process to decide on which **design** works better. It works by repeating the training process from scratch for k times, each with a different composition of the training and test sets. Because of that, you will have k models and k accuracy scores from their respective test set. You are not only interested in the average accuracy but also the standard deviation. The standard deviation tells whether the accuracy score is consistent or if some test set is particularly good or bad in a model.

Since k-fold cross validation trains the model from scratch a few times, it is best to wrap around the training loop in a function:

```
1 def model_train(X_train, y_train, X_test, y_test):
2     # create new model
3     model = nn.Sequential(
4         nn.Linear(8, 12),
5         nn.ReLU(),
6         nn.Linear(12, 8),
7         nn.ReLU(),
8         nn.Linear(8, 1),
9         nn.Sigmoid()
10    )
11
12    # loss function and optimizer
13    loss_fn = nn.BCELoss() # binary cross entropy
14    optimizer = optim.Adam(model.parameters(), lr=0.0001)
15
16    n_epochs = 25 # number of epochs to run
17    batch_size = 10 # size of each batch
18    batches_per_epoch = len(X_train) // batch_size
19
20    for epoch in range(n_epochs):
21        with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0, disable=True) as bar:
22            bar.set_description(f"Epoch {epoch}")
23            for i in bar:
24                # take a batch
25                start = i * batch_size
26                X_batch = X_train[start:start+batch_size]
27                y_batch = y_train[start:start+batch_size]
28                # forward pass
29                y_pred = model(X_batch)
30                loss = loss_fn(y_pred, y_batch)
31                # backward pass
32                optimizer.zero_grad()
33                loss.backward()
34                # update weights
35                optimizer.step()
36                # print progress
37                acc = (y_pred.round() == y_batch).float().mean()
38                bar.set_postfix(
39                    loss=float(loss),
40                    acc=float(acc)
41                )
42    # evaluate accuracy at end of training
43    y_pred = model(X_test)
44    acc = (y_pred.round() == y_test).float().mean()
45    return float(acc)
```

The code above is deliberately not printing anything (with `disable=True` in `tqdm`) to keep the screen less cluttered.

Also from scikit-learn, you have a function for k-fold cross validation. You can make use of it to produce a robust estimate of model accuracy:

```
1 from sklearn.model_selection import StratifiedKFold
2
3 # define 5-fold cross validation test harness
4 kfold = StratifiedKFold(n_splits=5, shuffle=True)
5 cv_scores = []
6 for train, test in kfold.split(X, y):
7     # create model, train, and get accuracy
8     acc = model_train(X[train], y[train], X[test], y[test])
9     print("Accuracy: %.2f" % acc)
10    cv_scores.append(acc)
11 # evaluate the model
12 print("%.2f%% (+/- %.2f%%)" % (np.mean(cv_scores)*100, np.std(cv_scores)*100))
```

Running this prints:

```
1 Accuracy: 0.64
```



```

2 Accuracy: 0.67
3 Accuracy: 0.68
4 Accuracy: 0.63
5 Accuracy: 0.59
6 64.05% (+/- 3.30%)

```

In scikit-learn, there are multiple k-fold cross validation functions, and the one used here is stratified k-fold. It assumes y are class labels and takes into account of their values such that it will provide a balanced class representation in the splits.

The code above used $k = 5$ or 5 splits. It means splitting the dataset into five equal portions, picking one of them as the test set and combining the rest into a training set. There are five ways of doing that, so the for-loop above will have five iterations. In each iteration, you call the `model_train()` function and obtain the accuracy score in return. Then you save it into a list, which will be used to calculate the mean and standard deviation at the end.

The `kfold` object will return to you the **indices**. Hence you do not need to run the train-test split in advance but use the indices provided to extract the training set and test set on the fly when you call the `model_train()` function.

The result above shows the model is moderately good, at 64% average accuracy. And this score is stable since the standard deviation is at 3%. This means that most of the time, you expect the model accuracy to be 61% to 67%. You may try to change the model above, such as adding or removing a layer, and see how much change you have in the mean and standard deviation. You may also try to increase the number of epochs used in training and observe the result.

The mean and standard deviation from the k-fold cross validation is what you should use to benchmark a model design.

Tying it all together, below is the complete code for k-fold cross validation:

```

1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 import tqdm
6 from sklearn.model_selection import StratifiedKFold
7
8 data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
9 X = data[:, 0:8]
10 y = data[:, 8]
11 X = torch.tensor(X, dtype=torch.float32)
12 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
13
14 def model_train(X_train, y_train, X_test, y_test):
15     # create new model
16     model = nn.Sequential(
17         nn.Linear(8, 12),
18         nn.ReLU(),
19         nn.Linear(12, 8),
20         nn.ReLU(),
21         nn.Linear(8, 1),
22         nn.Sigmoid()
23     )
24
25     # loss function and optimizer
26     loss_fn = nn.BCELoss() # binary cross entropy
27     optimizer = optim.Adam(model.parameters(), lr=0.0001)
28
29     n_epochs = 25 # number of epochs to run
30     batch_size = 10 # size of each batch
31     batches_per_epoch = len(X_train) // batch_size
32
33     for epoch in range(n_epochs):
34         with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0, disable=True) as bar:
35             bar.set_description(f"Epoch {epoch}")
36             for i in bar:
37                 # take a batch
38                 start = i * batch_size
39                 X_batch = X_train[start:start+batch_size]
40                 y_batch = y_train[start:start+batch_size]
41                 # forward pass
42                 y_pred = model(X_batch)
43                 loss = loss_fn(y_pred, y_batch)
44                 # backward pass
45                 optimizer.zero_grad()
46                 loss.backward()
47                 # update weights
48                 optimizer.step()
49                 # print progress
50                 acc = (y_pred.round() == y_batch).float().mean()
51                 bar.set_postfix(
52                     loss=float(loss),
53                     acc=float(acc)
54                 )
55             # evaluate accuracy at end of training
56             y_pred = model(X_test)
57             acc = (y_pred.round() == y_test).float().mean()
58             return float(acc)
59
60 # define 5-fold cross validation test harness
61 kfold = StratifiedKFold(n_splits=5, shuffle=True)
62 cv_scores = []
63 for train, test in kfold.split(X, y):

```

```

64 # create model, train, and get accuracy
65 acc = model_train(X[train], y[train], X[test], y[test])
66 print("Accuracy: %.2f" % acc)
67 cv_scores.append(acc)
68 # evaluate the model
69 print("%.2f%% (+/- %.2f%%)" % (np.mean(cv_scores)*100, np.std(cv_scores)*100))

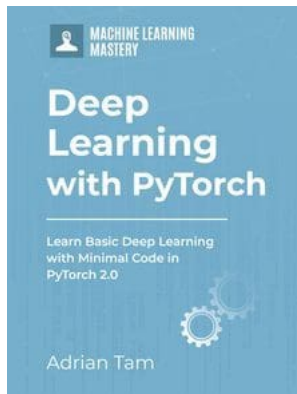
```

Summary

In this post, you discovered the importance of having a robust way to estimate the performance of your deep learning models on unseen data, and you learned how to do that. You saw:

- How to split data into training and test sets using scikit-learn
- How to do k-fold cross validation with the help of scikit-learn
- How to modify the training loop in a PyTorch model to incorporate test set validation and cross validation

Get Started on Deep Learning with PyTorch!



Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:
Deep Learning with PyTorch

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

Kick-start your deep learning journey with hands-on exercises

SEE WHAT'S INSIDE

Share

Share

Post

More On This Topic



Evaluate the Performance of Deep Learning Models in Keras



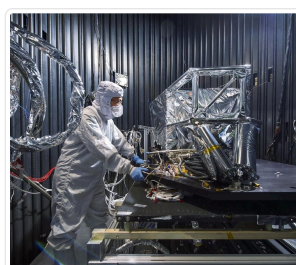
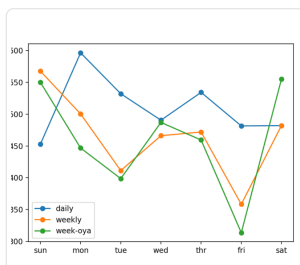
Evaluate the Performance of Machine Learning...



How to Evaluate Gradient Boosting Models with...



How to Evaluate the Skill of Deep Learning Models





About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.
[View all posts by Adrian Tam](#) →

< Creating a Training Loop for PyTorch Models

Building a Multiclass Classification Model in PyTorch >

6 Responses to *How to Evaluate the Performance of PyTorch Models*

- 

Oladimeji February 10, 2023 at 5:54 pm #

REPLY ↩

Thanks a lot for your efforts highly appreciated
- 

James Carmichael February 11, 2023 at 7:38 am #

REPLY ↩

You are very welcome Oladimeji! We appreciate your support and feedback!
- 

Jahangir Khan July 25, 2023 at 12:40 am #

REPLY ↩

Good Job ! Thanks James
- 

James Carmichael July 25, 2023 at 8:31 am #

REPLY ↩

You are very welcome Jahangir! We greatly appreciate the support and feedback!
- 

shadow February 2, 2024 at 5:08 pm #

REPLY ↩

Thanks James !
- 

James Carmichael February 3, 2024 at 9:44 am #

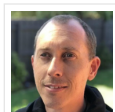
REPLY ↩

You are very welcome! We appreciate your support!

Leave a Reply

Name (required)

Email (will not be published) (required)

[SUBMIT COMMENT](#)**Welcome!**I'm *Jason Brownlee* PhDand I **help developers** get results with **machine learning**.[Read more](#)**Never miss a tutorial:****Picked for you:**[PyTorch Tutorial: How to Develop Deep Learning Models with Python](#)[LSTM for Time Series Prediction in PyTorch](#)[Deep Learning with PyTorch \(9-Day Mini-Course\)](#)[Calculating Derivatives in PyTorch](#)[Develop Your First Neural Network with PyTorch, Step by Step](#)

Loving the Tutorials?

The **Deep Learning with PyTorch** EBook
is where you'll find the **Really Good** stuff.

[>> SEE WHAT'S INSIDE](#)

Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.



